# Letting an LLM investigate a live TorQ-ops stack - safely

*The question: when the LLM writes code or queries to investigate a prod incident, what stops it harming the box - OOM-ing it, deleting data, killing a tickerplant?*

## First - the only thing that really matters: it can't get around the gate

Every control below works only because **the agent has one wire out (the gate) and the kernel enforces it.** This is not a rule the agent chooses to follow - the operating system has physically closed every other path:

| The agent tries to... | Why it can't |
|---|---|
| Open a raw socket to a prod process | empty **network namespace** - no route to prod exists; only the gate is reachable |
| Run a shell, or q itself | no shell/q binary in the **read-only rootfs**; **seccomp** blocks the syscalls |
| Write files, or load native code (`2:`) | read-only filesystem + **Landlock**; seccomp blocks module/`.so` loads |
| Edit the policy, or disable the gate | policy is **Ed25519-signed + read-only**, and the decision runs **out-of-process** - not a variable the agent can overwrite (unlike an in-process `.z.pg` guard) |
| Pretend to be a different user | identity comes from the **authenticated channel**, never the request |

**The one thing you must not ship: an agent with a raw shell or direct IPC to prod.** That is the only way this breaks. Everything else rests on confinement making "only through the gate" physically true.

## Then, within that box, Aegis handles both kinds of query

The agent has two ways to query - and **both go through Aegis, which controls quality and safety in one gate.** No separate tool, no dependency on a code editor or hook (this is an API agent).

- **Pre-built / structured tools (the normal path).** The agent sends a structured

request and the compiler emits the only q that runs - date-first, allowlisted, **time-bounded and capped** (so a diagnostic can't OOM the box), with a mandatory per-principal row filter. Dangerous ops (`system`, `delete`, `2:`, `exit`) aren't in the grammar - **inexpressible.**

- **Hand-written q (when a diagnostic needs it).** Aegis does **not** scan the q text for

bad patterns (a losing game). It **parses** the q, accepts only the **safe subset**, and **recompiles it through the same compiler** - so it comes back date-first, capped, and entitlement-filtered, exactly like the structured path. The agent's raw q is **never executed**; anything outside the safe subset (`system`, `value`, a second statement, an unknown function) is **rejected**. A query written date-*second* even comes back recompiled date-*first* - the OOM foot-gun is fixed, not just flagged.

Around both: restarts need human approval, touching the tickerplant or gateway is never allowed, and every decision is signed, fail-closed, and WORM-audited.

## Honest boundary

The strong guarantee is the structured path: dangerous and unbounded q are *inexpressible*. Hand-written q is governed by **allowlist-on-parse** - we run only q we can re-derive as a safe structured query, and reject the rest - which is far stronger than a pattern denylist, but the recognised grammar is a **curated subset that grows**, so some legitimate-but-exotic q is rejected (safe) and goes to break-glass. It stays wrapped by caps + confinement regardless. Proven where provable (exhaustive over the modeled space, Z3, Cedar), 0/50 red-team attack-success on a real 4-billion-row estate.

> **Bottom line:** the agent's only route to prod is a gate it can't see around, and the kernel makes that true. Through that gate, every query - pre-built or hand-written - is recompiled by Aegis into safe, bounded, entitled q before it runs. It can't OOM the box, can't run a destructive command, and can't act outside policy - by construction, not by good behaviour.